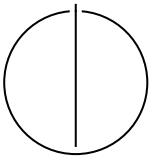
SCHOOL OF COMPUTATION, INFORMATION AND TECHNOLOGY — INFORMATICS

TECHNISCHE UNIVERSITÄT MÜNCHEN

Bachelor's Thesis in Informatics

Comparative Analysis of Widely Used Zipfian Distribution Implementations

Rodrigo Felix Forno



SCHOOL OF COMPUTATION, INFORMATION AND TECHNOLOGY — INFORMATICS

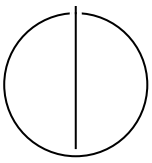
TECHNISCHE UNIVERSITÄT MÜNCHEN

Bachelor's Thesis in Informatics

Comparative Analysis of Widely Used Zipfian Distribution Implementations

Vergleichende Analyse von weit verbreiteten Zipfian Distribution Implementierungen

Author:	Rodrigo Felix Forno
Examiner:	Prof. Dr. Viktor Leis
Supervisor:	MSc. Bohyun Lee, MSc. Gabriel Haas
Submission Date:	Submission date



I confirm that this bachelor's thesis is my own work and I have documented all sources and material used.

Munich, Submission date Rodrigo Felix Forno

Acknowledgments

Abstract

Contents

Acknowledgments **iv**

Abstract **v**

1 Introduction **1**

2 Preliminaries **3**
2.1 The Zipfian Distribution 3
2.2 Recognizing a Zipfian distribution 4
2.3 Discrete Variate Generation 4

3 Approach **7**
3.1 Research Approach 7
3.2 Common Frameworks 8
3.2.1 Yahoo! Cloud Serving Benchmark 9
3.2.2 Flexible I/O Tester 10
3.2.3 Sysbench 11
3.2.4 OLTP-Bench 11

4 Evaluation **12**
4.1 Experimental Setup 12
4.2 Methodology 12
4.2.1 Evaluating accuracy 14
4.3 Evaluating Performance 14
4.3.1 Microbenchmarking 15
4.3.2 Macrobenchmarking 17

Abbreviations **18**

List of Figures **19**

List of Tables **20**

Bibliography **21**

1 Introduction

The Zipfian distribution is a power law that has been observed in a wide range of natural and artificial phenomena. It is characterized by the fact that the frequency of an element is inversely proportional to its rank, meaning the position of the element in the sorted list of elements. A quick example makes this principle quite clear.

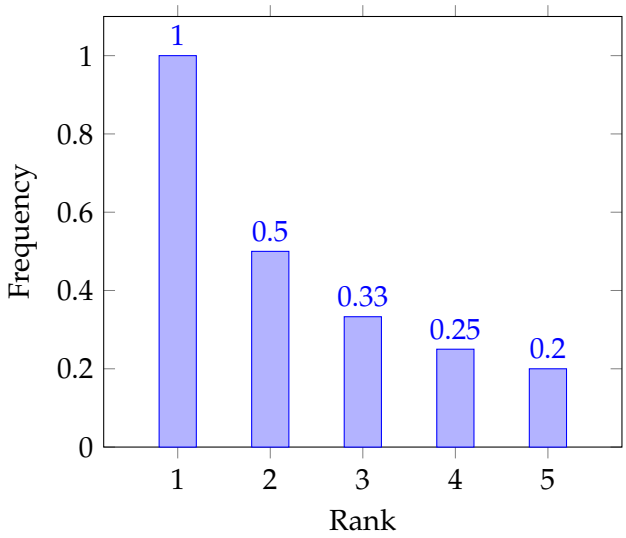


Figure 1.1: Example of a Zipfian distribution

This means that the most frequent element is twice as frequent as the second most frequent element, three times as frequent as the third most frequent element, and so on. This distribution has been observed in a wide range of natural phenomena, such as the distribution of words in natural languages, the distribution of cities by population, among others [1]. And also within the fields of storage systems research, networking theory as well as databases, there has been more compelling evidence that the Zipfian distribution (or more generally, heavy-tailed distributions) is a good model for the distribution of access patterns in these systems [2]–[4].

The focus however of this thesis is on the large-scale generation of Zipfian samples, which are essential for the evaluation of latency and general performance in storage systems and databases. To this end, the state-of-the-art in synthetic workload generation

is reviewed and common solutions are identified and evaluated. Parallely, a study of variate generation algorithms is conducted, with a focus on the generation of Zipfian variates. This will allow for the comparison in terms of speed and accuracy of common implementations as well as give a baseline for an improved version.

The main contributions of this thesis are:

- A review of the state-of-the-art in synthetic workload generation, with a focus on the generation of Zipfian variates.
- A study of variate generation algorithms, with a focus on the generation of Zipfian variates.
- An implementation of a Zipfian variate generator in C++20, with a focus on performance and algorithmic simplicity
- An evaluation of the performance of the implemented Zipfian variate generator in comparison to existing solutions

2 Preliminaries

Before proceeding with the approach, it's important to give a precision definition of

1. the Zipfian distribution
2. Recognizing a zipfian distribution
3. Discrete Variate generation

2.1 The Zipfian Distribution

This distributions has had ample impact in the areas of quantitative linguistics and information theory [5], as well as within computer science in the fields of storage systems and Relational Database Management System (RDBMS)[2]. It states that "(...) when a list when a list of measured values is sorted in decreasing order, the value of the n-th entry is often approximately inversely proportional to n" [6]. Mathematically, this is expressed by

$$\textbf{word frequency} \propto \frac{1}{\textbf{word rank}} \tag{2.1}$$

The form of interest for this research has an additional parameter α, yielding

$$\textbf{frequency} \propto \frac{1}{\textbf{rank}^{\alpha}} \tag{2.2}$$

The probability distribution for a given rank k given N elements is:

$$f(k, N, \alpha) := \begin{cases} \frac{1}{H_{N,\alpha}} \frac{1}{k^{\alpha}} & 1 \leq k \leq N \\ 0 & k < 1 \text{ or } N < k \end{cases} \tag{2.3}$$

where $H_{N,\alpha}$ is defined by the generalized harmonic number

$$H_{N,\alpha} = \sum_{k=1}^{N} \frac{1}{k^{\alpha}} \tag{2.4}$$

To give more concrete form to this distribution, we can visualize how the curve looks like for $N = 100$ and with an exponent $\alpha = 1$

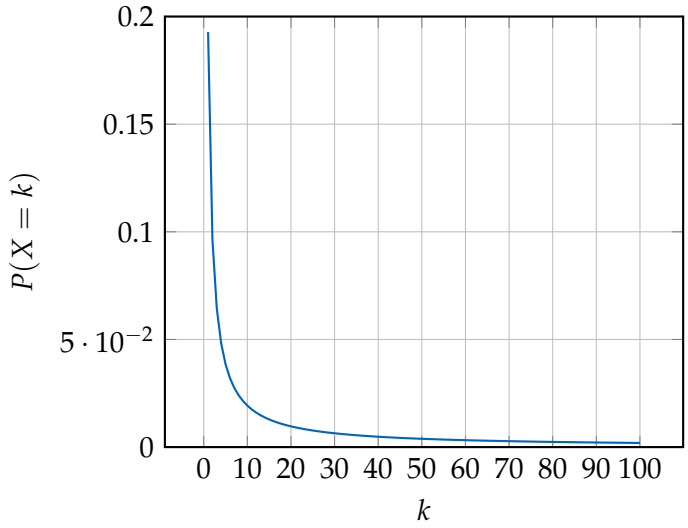


Figure 2.1: Example of a Zipfian distribution with parameters $N, \alpha = (100, 1)$

2.2 Recognizing a Zipfian distribution

One way to visually recognize whether a sample follows a Zipfian distribution or not is by using a plot in log-log scale. This is justified by taking equation 2.3 (assuming the probability is always well-defined) and applying a log on both sides.

$$\log f(k, N, \alpha) = -\alpha \log k - \log(H_{N,\alpha}) \tag{2.5}$$

Since this is clearly a linear form: $Y = mX + C$, we should expect data that was generated according to a Zipfian distribution to have a linear form when transformed with the logarithm. Applying this on the last graph yields Figure 2.2.

It's a useful criterium for graphically evaluating if the Zipfian has the expected behaviour through the whole domain. It's not foolproof since other power laws also turn linear when transformed with the Zipfian, but since all generators analyse *try* to generate such a distribution, the problem disappears.

2.3 Discrete Variate Generation

Theoretically defining a distribution is one aspect of the work, but the main goal is to generate variates which follow a desired distribution. This can be represented by 2.6.

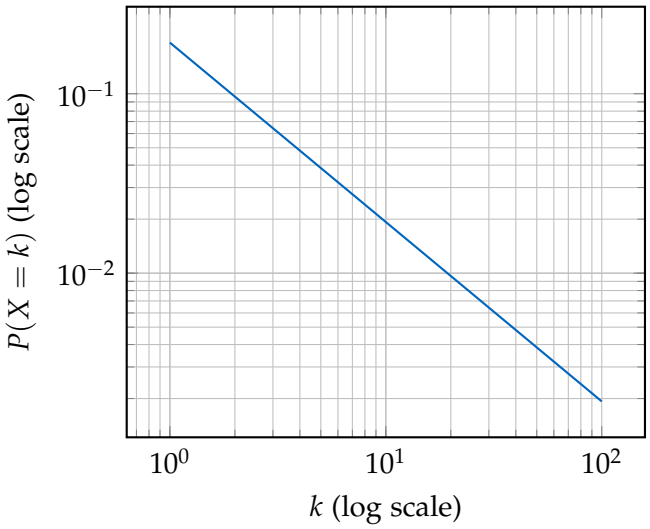


Figure 2.2: Application of log on data from 2.1

$$F(x) = Pr(X \leq x) \quad -\infty < x < \infty \tag{2.6}$$

The theory for developing variate generators rely on the following assumptions [7]:

- There exists a perfect uniform $(0, 1)$ generator that produces a sequence of *independent* random variables uniformly distributed in $(0, 1)$
- Computers can save and manipulate real numbers

Which are directly violated once implementation in digital computers is considered. While not *a priori* a problem, since computers always introduce different types of errors in numerical computations, it's important to consider by *how much* these assumptions are violated. Even more so if certain distributions exarcebate problems by this violation [8].

There are many possible implementations and theoretical frameworks for creating discrete samplers, be them exact or approximative. Each method has it's advantages and disadvantages and in the next sections, they will be evaluated. It's important to note, however, that other approaches are possible, but have been omitted since the ones present in the table have already compared more favourably to them (see [7], [9], [10]). Below follows a table (2.1) summarizing the methods which will be analysed later.

Method	Efficiency	Complexity	Best Use Cases
Rejection-Inversion [11]	Very fast ($O(1)$, best exact method)	More complex, efficient rejection	High-performance simulations, library implementations
Alias Method [10]	Extremely fast ($O(1)$ after $O(N)$ setup)	Needs precomputed tables, memory-intensive	When support is finite and large N is manageable
Marsaglia's Layered Rejection [12]	Very fast ($O(1)$, optimized for heavy tails)	Moderate, requires precomputed layer boundaries but low memory use	When many samples are needed efficiently, suitable for Zipf–Mandelbrot with infinite support
Continuous Approximation	Extremely fast ($O(1)$)	Simple, uses power function	Approximate Zipf-like sampling for large N

Table 2.1: Summary of methods for sampling from the Zipf-Mandelbrot distribution

3 Approach

The goal of this thesis is to identify the most widely used frameworks for generating zipfian synthetic data in the systems programming and database community and make a comparative evaluation. The first step required to achieve this is a systematic review of current approaches. The following sections describe

1. the **research approach** used to identify the frameworks,
2. the **common frameworks** recognized from the search along with short descriptions
3. the **popularity and impact** of those tools

Afterwards, the **evaluation criteria** used to compare the frameworks are presented and clarified, along with a presentation of the tool used to compile those results.

3.1 Research Approach

To identify relevant frameworks used by the systems programming and database community, first the scope of venues and journals was defined. The list is comprised of:

- Proceedings of the VLDB Endowment (PVLDB)
- ACM Special Interest Group on Management of Data (SIGMOD)
- Conference on Innovative Data Systems Research (CIDR)
- USENIX Symposium on Operating Systems Design and Implementation (OSDI)
- USENIX Symposium on Networked Systems Design and Implementation (NSDI)
- USENIX Conference on File and Storage Technologies (FAST)

These conferences were chosen due to their high selectivity and special relevancy for storage technology research, where there is a prevalence of standardized IO benchmarks.

The search was conducted broadly following the steps indicated in [13], as defined below:

1. Word search on keyword search matrix (see table 3.1)
2. Identify the frameworks used in the papers
3. Extended keyword search, this time adding the frameworks identified in the previous step

The first step suggests the use of a keyword matrix. This works by choosing appropriate keywords which refer to the topic of interest. Then for each such keyword, synonyms are used to improve the accuracy and scope of the search. It's defined below as:

	Keyword 1	Keyword 2	Keyword 3
synonyms	zipf zipfian	benchmark workload evaluation empirical dataset	synthetic generated sampled sampler

Table 3.1: Keyword search matrix

The columns are the keywords which are boolean OR'ed, whereas the rows are boolean AND'ed. This means that at least one word of each column should appear in the search.

These queries where then run for the conferences and journals listed before on an unrestricted timescale. The databases consulted are

- IEEE Xplore
- ACM Digital Library
- DBLP
- Google Scholar

3.2 Common Frameworks

On a first phase, the following frameworks were identified among the top 20 results of each conference

Frameworks
Yahoo! Cloud Serving Benchmark (YCSB)
Flexible I/O Tester (FIO)
Sysbench
OLTP-Bench [14]
DBgen

These frameworks were then searched with the same keyword matrix and their name as a last keyword. This has yielded the following results:

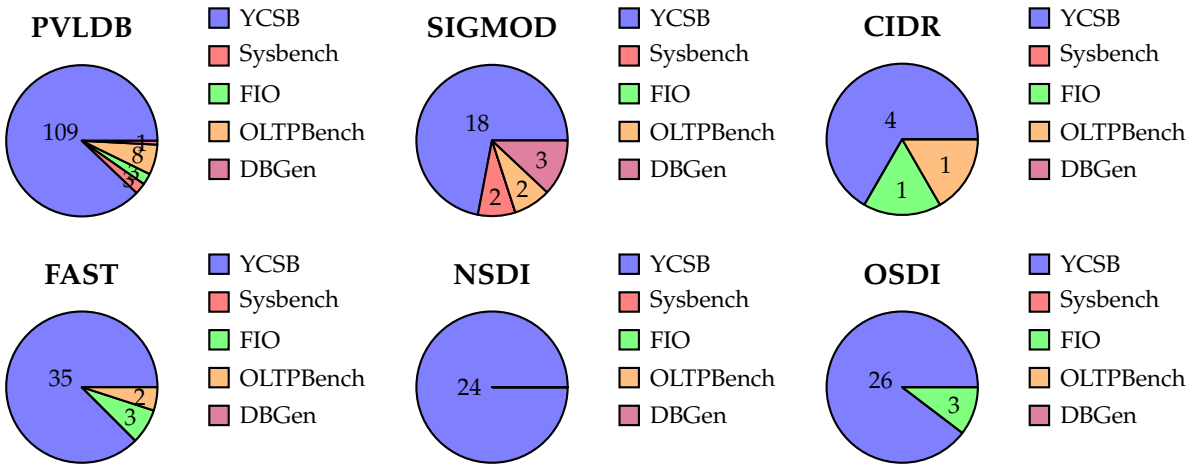


Figure 3.1: Pie charts for each conference (excluding Web Polygraph and pgBench).

The most popular framework by far is **YCSB**, then followed by **Sysbench** along with **FIO**. Other frameworks are not as prevalent, but have appeared in the search results.

3.2.1 Yahoo! Cloud Serving Benchmark

YCSB [15] is a widely used benchmarking tool for cloud databases. It was developed by *Yahoo! Research* and is now maintained by a community of developers. Its working principle is to generate workloads in a threaded environment, such that the databases can be also tested for concurrent accesses. It's also capable to loading tables with data organized according to different distributions, among which the Zipfian distribution.

To achieve this, YCSB uses a Zipfian generator which is based on the one proposed by Jim Gray [2]. One limiting factor on his original paper is that it's the generated keys are lexographically sorted, meaning the first rank (in the case of an integer, 0)

is the most frequent. To better reflect real workloads, the keys should be randomly distributed within the range. See figure 3.2 for a comparison of the two distributions.

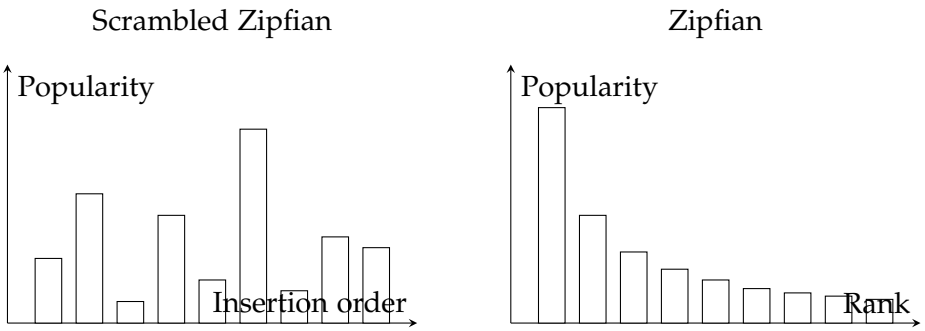


Figure 3.2: Comparison of Jim Gray's version (right) and the one employed by YCSB (left).

To go around this issue, YCSB uses a hashing algorithm to randomly spread the values in the range as well as employing an extended domain to reduce collisions [15].

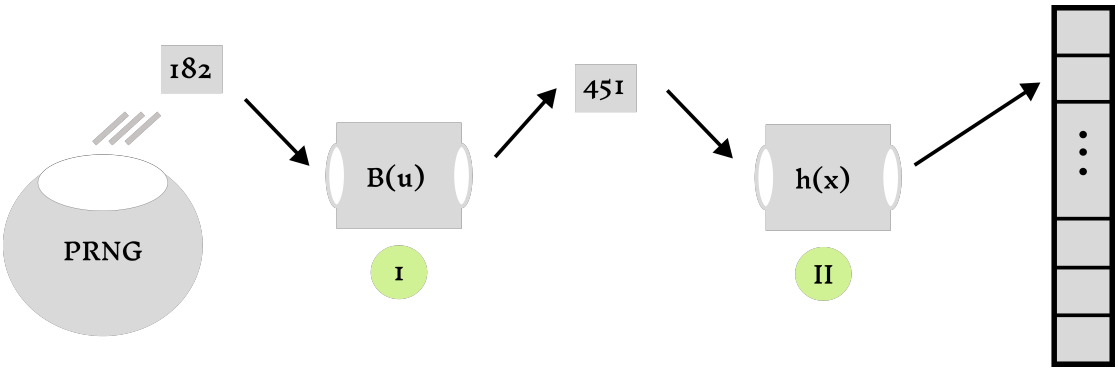


Figure 3.3: YCSB does sampling by inversion (1) and then hashes the value to spread in in the available range (2)

3.2.2 Flexible I/O Tester

FIO [16] is a benchmarking tool for storage systems. Much like YCSB, its purpose is generating workloads following a certain pattern access to test a storage system. The workloads can vary in type, including sequential or random and read-only, write-only or mixed. In the case of **random I/O** , the user is able to to change the access pattern with the goal of simulating both asymetrical and more symmetrical workloads. The underlying distributions include:

- Zipfian
- Normal
- Pareto
- Uniform

The generation of a Zipfian distribution works by the same principle as YCSB, meaning:

1. Generate a zipfian variate using the algorithm proposed by Jim Gray [2]
2. Hash the value to spread it in the range

The same principle applies as in 3.3

3.2.3 Sysbench

Sysbench [17] is a benchmarking tool for databases as well as storages systems. It works in a manner very similar to YCSB, using multi-threading to strain the system with concurrent accesses. The tool is capable of generating synthetic data according to different probability distributions, among which the Zipfian distribution.

What sets it apart from the previous ones is that the variate generation uses a different algorithm. The one employed by Sysbench is based on a paper by Wolfgang Hörmann [11] and the implementation is based off the Apache Commons Math library [18], under the name *RejectionInversionZipfSampler*.

3.2.4 OLTP-Bench

The OLTP-Bench [14] strives to unify different types of benchmarks for evaluating a given Database Management System (DBMS). It favours a modular design, enabling very different benchmarks to be used under a common interface, which makes it quite extensible. However, it also has enough benchmarks built-in to be used directly, among which:

- AuctionMark [19]
- Transaction Processing Performance Council Benchmark C (TPC-C)
- YCSB

The focus will be rather layed upon the AuctionMark benchmark as it's the one excepting YCSB which generates synthetic data following a Zipfian distribution. The implementation is based on an inverse transformation sampling sped up by a binary search.

4 Evaluation

In the previous chapters, some variate generators included in common frameworks were briefly presented, as well as the underlying algorithms used to sampled the Zipfian distribution. To make a fair comparison, it's necessary to evaluate the accuracy of the distributions generated, as well as the performance of the algorithms used to generate them. Additionally, simplicity of the methods will be considered.

- **Accuracy**: The statistical properties of the generated distributions will be compared to the expected values given by the theoretical Zipfian distribution
- **Performance**: The time taken to generate the distributions will be measured and compared using standard profilling tools as well as custom-written benchmarks
- **Simplicity**: The complexity of the algorithms used to generate the distributions will be compared, as well as the ease of use of the libraries that implement them

The variate generators considered for this approach are exactly those presented in the previous chapters: YCSB, FIO, Sysbench and OLTP-Bench, along with implementations of the algorithms discussed in 3.

4.1 Experimental Setup

This section describes the hardware and software environment used to conduct the experiments.

All experiments described in this chapter were conducted on a machine with the specifications detailed in table 4.1. The software environment used is detailed in table 4.2. Since the samplers were implemented in different languages, it's also necessary to detail the versions of the interpreters used to run them.

4.2 Methodology

Evaluations were conducted both for accuracy and performance. In the first case, different criteria will be used to give a more rounded view of the quality of the generated distributions. The general flow is:

CPU Model	Clockspeed	Number of Cores	Memory Capacity	Memory Speed
AMD Ryzen 7 5850U	3.6 GHz	8	16 GB	3200 MHz

Table 4.1: Computer Specifications

Operating System	Kernel Version	Relevant Tools
Debian GNU/Linux 12	6.1.0-amd64	perf[20], JMH [21], Google Benchmark [22]

Table 4.2: Software Specification

Language/Version	Compiler Benchmark/Sampler	
C++20	gcc 12.2.0	Own implementation
Java 17	OpenJDK 17.0.8	YCSB
C (GNU17)	gcc 12.2.0	Sysbench, FIO
Rust	1.58.0	None

Table 4.3: Interpreter Versions

1. Generate a Zipfian distribution with the developed tool as explained in 3 and store it appropriately
2. Use whole-distribution measures, log-graphs and other tools to evaluate and compare the accuracy of the samples
3. Repeat the process for different parameters of the Zipfian distribution

For performance, the main measure of interest is throughput - the amount of variates generated in a time interval (e.g. $10M/ms$). Since benchmarking in modern systems is a rife with pitfalls [21], [23], some precautions should be taken before running the code. More details follow in the coming sections.

4.2.1 Evaluating accuracy

Defining accuracy is domain-dependent and the choice of accuracy measures is conditional on which statistical properties are of interest. The starting point will be a general measure that can be used to evaluate how well the generated distributions compare to the theoretical one over the whole body of the distribution. This is akin to defining a measure that takes two distributions, which says how well one matches the other. Graphically:

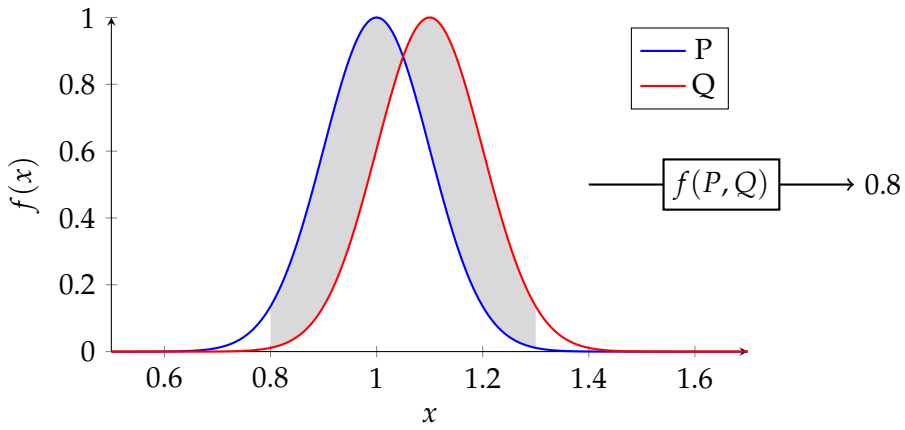


Figure 4.1: Accuracy Measure

In the context of research in storage systems and databases, there is a specific interest that the hotpaths are accurately represented, as well as the tail of the distribution ([3], [24]), since cold paths could increase latency measures. There are some indications that self-similar distributions only very slowly converge to their theoretical values [25] when sampling, but there are no concrete guidelines on how to evaluate the accuracy of non-uniform variate generation. Some advice follows from [26], where the authors suggest that the best way to evaluate the fit of a dataset to a power-law distribution is to use the Kolmogorov-Smirnov test. This is equivalent to the starting point explained above. However, as will be explained later, this might have some drawbacks when applied to the considered use case and is the reason other approaches will also be used.

4.3 Evaluating Performance

As already mentioned, evaluating performance is a tricky business. The main measure of interest is throughput, which is the amount of variates generated in a time interval. On the one hand, it's important not to let undesired factors influence the results,

which means microbenchmarking - running only the code routines of interest - is necessary. On the other hand, it's also important to capture realistic scenarios where other processes might be running parallely to the sampling procedure, as well as the fact that the results of sampling will be used as indexes/keys for storage systems and databases. Therefore it's important to also benchmark runs that include the necessary I/O present in real-world scenarios.

4.3.1 Microbenchmarking

Microbenchmarking is the process of measuring the performance of small code snippets, usually in the order of microseconds. The main goal is to measure the performance of the code in isolation, without the influence of other factors. Since the languages of interest are primarily C++, C and Java (see table 4.3), the tools used to measure performance will be Google Benchmark [22] for C++ and C, and JMH [21] for Java. For C++ and C applications, the following precations will be taken (for instruction on how to accomplish the following, see [27]):

- Set specific CPU core for the process
- Disable frequency scaling
- Disable turbo boost
- Limit the available memory to the process
- Warm-up runs

At last, it's important to define the runs. Since what is going to be measured is throughput, we define the duration of a single run to be 1 second. This will be repeated 30 times and the average throughput will be calculated. Since the speed is invariant to domain size and distribution parameter, these are kept fixed at 10^8 and 0.8, respectively.

For the Google Benchmark, the following command will be used to run the benchmarks (for more details on how to use the tool, see [22]):

```
./benchmark --benchmark_repetitions=30 --benchmark_min_warmup_time=1.0
```

As a precaution for the sample generation not be optimized away, the function `benchmark::DoNotOptimize` on the results of the computation was used. Similarly for the JMH (more details on [21], [28]):

```
java -jar target/benchmarks.jar -bm thrpt -wi 10 -i 30 -f 1 -r 1
```

The results depende heavily on which JVM is used, but some common patterns exist to avoid making code be optimized away:

- Use the `Blackhole` object to consume the results of the computation
- Use the `@CompilerControl` annotation to avoid inlining
- Use the `@State` annotation to avoid dead code elimination

This procedure yielded the following results:

Sampler	Average Throughput (M/ms)	Standard Deviation
FIO	26.2	0.01
ApacheCommons	27.4	0.07
YCSB	29.9	0.14
Sysbench	43.6	0.15

Table 4.4: Microbenchmarking Results

From the table, we can recognize that the Sysbench sampler is the fastest, which is to be expected given the overhead of the JVM. But to further investigate why this difference is present, a run with a profiler is required. Running `perf`[20] along with the JMH tool yields the following results for YCSB and ApacheCommons (only relevant lines):

ApacheCommons

```
Baseline:  565.140.532.065 instructions 4,33 insn per cycle
Sampler:  204.065.670.512 instructions 1,56 insn per cycle
Sampler:  11.310.090 L1-dcache-load-misses 0,02% of all L1-dcache
accesses
```

YCSB

```
Baseline:  578.884.719.478 instructions 4,43 insn per cycle
Sampler :  392.403.334.414 instructions 2,92 insn per cycle
Sampler:  339.375.866 L1-dcache-load-misses 0,43 % of all L1-dcache
accesses
Sampler:  789.273 L1-icache-load-misses 0,30% of all L1-icache
accesses
```

Given that both the baseline and the actual sample routine have similarly high IPC and low number of data and instruction cache misses, the lower throughput in Java is likely not caused by an inefficient implementatation, but rather by the overhead inherent to the JVM runtime, such as garbage collection, memory management and JIT compilation. These aspects won't be explored further, which limits a rigorous conclusion. Nonetheless this points towards the JVM being responsible for the difference in performance.

4.3.2 Macrobenchmarking

To run macrobenchmarks, the tool explained in Chapter 3 will be used to generate the distributions and load them onto a DuckDB instance [29]. The tool will be run with the same parameters as the microbenchmarking runs, and the time taken to generate the distributions will be measured. With the exception of warm-up runs, the same precautions will be taken as in the microbenchmarking runs (see 4.3.1). Since the use case of these samplers is to generate keys for storage systems and databases, the ranges considered for the support size are between 10^8 and 10^{10}, and the skewness will be varied between 0.5 and 1.5.

Abbreviations

PVLDB Proceedings of the VLDB Endowment

SIGMOD ACM Special Interest Group on Management of Data

CIDR Conference on Innovative Data Systems Research

OSDI USENIX Symposium on Operating Systems Design and Implementation

NSDI USENIX Symposium on Networked Systems Design and Implementation

FAST USENIX Conference on File and Storage Technologies

YCSB Yahoo! Cloud Serving Benchmark

FIO Flexible I/O Tester

DBMS Database Management System

TPC-C Transaction Processing Performance Council Benchmark C

RDBMS Relational Database Management System

List of Figures

1.1 Example of a Zipfian distribution 1

2.1 Example of a Zipfian distribution with parameters $N, \alpha = (100, 1)$ . . . 4
2.2 Application of log on data from 2.1 5

3.1 Pie charts for each conference (excluding Web Polygraph and pgBench). 9
3.2 Comparison of Jim Gray's version (right) and the one employed by YCSB (left). 10
3.3 YCSB does sampling by inversion (1) and then hashes the value to spread in in the available range (2) 10

4.1 Accuracy Measure 14

List of Tables

2.1 Summary of methods for sampling from the Zipf-Mandelbrot distribution 6

3.1 Keyword search matrix 8

4.1 Computer Specifications 13
4.2 Software Specification 13
4.3 Interpreter Versions 13
4.4 Microbenchmarking Results 16

Bibliography

[1] B. B. Mandelbrot, "Is Nature Fractal?" *Science*, vol. 279, no. 5352, pp. 783–783, Feb. 6, 1998. DOI: 10.1126/science.279.5352.783c.

[2] J. Gray, P. Sundaresan, S. Englert, K. Baclawski, and P. J. Weinberger, "Quickly generating billion-record synthetic databases," *SIGMOD Rec.*, vol. 23, no. 2, pp. 243–252, May 24, 1994, ISSN: 0163-5808. DOI: 10.1145/191843.191886.

[3] M. E. Crovella, "Performance Evaluation with Heavy Tailed Distributions," in *Computer Performance Evaluation.Modelling Techniques and Tools*, B. R. Haverkort, H. C. Bohnenkamp, and C. U. Smith, Eds., Berlin, Heidelberg: Springer, 2000, pp. 1–9, ISBN: 978-3-540-46429-7. DOI: 10.1007/3-540-46429-8_1.

[4] J. Dean and L. A. Barroso, "The tail at scale," *Communications of the ACM*, vol. 56, no. 2, pp. 74–80, Feb. 2013, ISSN: 0001-0782, 1557-7317. DOI: 10.1145/2408776.2408794.

[5] D. M. W. Powers, "Applications and explanations of Zipf's law," in *Proceedings of the Joint Conferences on New Methods in Language Processing and Computational Natural Language Learning - NeMLaP3/CoNLL '98*, Sydney, Australia: Association for Computational Linguistics, 1998, p. 151, ISBN: 978-0-7258-0634-7. DOI: 10.3115/1603899.1603924.

[6] *Zipf's law*, in *Wikipedia*, Feb. 12, 2025.

[7] M. E. Kuhl, "History of random variate generation," in *Proceedings of the 2017 Winter Simulation Conference*, ser. WSC '17, Las Vegas, Nevada: IEEE Press, Dec. 3, 2017, pp. 1–12, ISBN: 978-1-5386-3427-1.

[8] F. Radicchi, "Underestimating extreme events in power-law behavior due to machine-dependent cutoffs," *Physical Review E*, vol. 90, no. 5, p. 050 801, Nov. 3, 2014, ISSN: 1539-3755, 1550-2376. DOI: 10.1103/PhysRevE.90.050801. arXiv: 1405.0058 [physics].

[9] L. Devroye, "Discrete Univariate Distributions," in *Non-Uniform Random Variate Generation*, L. Devroye, Ed., New York, NY: Springer, 1986, pp. 485–553, ISBN: 978-1-4613-8643-8. DOI: 10.1007/978-1-4613-8643-8_10.

[10] R. A. Kronmal and A. V. Peterson, "On the Alias Method for Generating Random Variables from a Discrete Distribution," *The American Statistician*, vol. 33, no. 4, pp. 214–218, Nov. 1979, ISSN: 0003-1305, 1537-2731. DOI: 10.1080/00031305.1979.10482697.

[11] W. Hörmann and G. Derflinger, "Rejection-inversion to generate variates from monotone discrete distributions," *ACM Transactions on Modeling and Computer Simulation*, vol. 6, no. 3, pp. 169–184, Jul. 1996, ISSN: 1049-3301, 1558-1195. DOI: 10.1145/235025.235029.

[12] G. Marsaglia, W. W. Tsang, and J. Wang, "Fast Generation of Discrete Random Variables," *Journal of Statistical Software*, vol. 11, no. 3, 2004, ISSN: 1548-7660. DOI: 10.18637/jss.v011.i03.

[13] W. M. Bramer, G. B. de Jonge, M. L. Rethlefsen, F. Mast, and J. Kleijnen, "A systematic approach to searching: An efficient and complete method to develop literature searches," *Journal of the Medical Library Association : JMLA*, vol. 106, no. 4, pp. 531–541, Oct. 2018, ISSN: 1536-5050. DOI: 10.5195/jmla.2018.283. pmid: 30271302.

[14] D. E. Difallah, A. Pavlo, C. Curino, and P. Cudre-Mauroux, "OLTP-Bench: An extensible testbed for benchmarking relational databases," *Proc. VLDB Endow.*, vol. 7, no. 4, pp. 277–288, Dec. 1, 2013, ISSN: 2150-8097. DOI: 10.14778/2732240.2732246.

[15] B. F. Cooper, A. Silberstein, E. Tam, R. Ramakrishnan, and R. Sears, "Benchmarking cloud serving systems with YCSB," in *Proceedings of the 1st ACM Symposium on Cloud Computing*, ser. SoCC '10, New York, NY, USA: Association for Computing Machinery, Jun. 10, 2010, pp. 143–154, ISBN: 978-1-4503-0036-0. DOI: 10.1145/1807128.1807152.

[16] "1. fio - Flexible I/O tester rev. 3.38 — fio 3.38-19-gd1eb documentation." (), [Online]. Available: https://fio.readthedocs.io/en/latest/fio_doc.html (visited on 01/16/2025).

[17] A. Kopytov, *Akopytov/sysbench*, Jan. 16, 2025.

[18] "Math – Commons Math: The Apache Commons Mathematics Library." (), [Online]. Available: https://commons.apache.org/proper/commons-math/ (visited on 02/22/2025).

[19] "AuctionMark: An OLTP Benchmark for Shared-Nothing Database Management Systems « H-Store." (), [Online]. Available: https://hstore.cs.brown.edu/projects/auctionmark/ (visited on 02/22/2025).

[20] "Perf: Linux profiling with performance counters." (), [Online]. Available: https://perfwiki.github.io/main/ (visited on 03/09/2025).

[21] "Avoiding Benchmarking Pitfalls on the JVM." (), [Online]. Available: https://www.oracle.com/technical-resources/articles/java/architect-benchmarking.html (visited on 03/09/2025).

[22] "Google/benchmark: A microbenchmark support library." (), [Online]. Available: https://github.com/google/benchmark (visited on 03/09/2025).

[23] D. Beyer, S. Löwe, and P. Wendler, "Benchmarking and Resource Measurement," in *Model Checking Software*, B. Fischer and J. Geldenhuys, Eds., Cham: Springer International Publishing, 2015, pp. 160–178, ISBN: 978-3-319-23404-5. DOI: 10.1007/978-3-319-23404-5_12.

[24] "The Tail at Scale." (), [Online]. Available: https://research.google/pubs/the-tail-at-scale/ (visited on 03/10/2025).

[25] M. E. Crovella and L. Lipsky, "Simulations with Heavy-Tailed Workloads," in *Self-Similar Network Traffic and Performance Evaluation*, John Wiley & Sons, Ltd, 2000, pp. 89–100, ISBN: 978-0-471-20644-6. DOI: 10.1002/047120644X.ch3.

[26] A. Clauset, C. R. Shalizi, and M. E. J. Newman, "Power-law distributions in empirical data," *SIAM Review*, vol. 51, no. 4, pp. 661–703, Nov. 4, 2009, ISSN: 0036-1445, 1095-7200. DOI: 10.1137/070710111. arXiv: 0706.1062 [physics].

[27] "Benchmarking tips — LLVM 21.0.0git documentation." (), [Online]. Available: https://llvm.org/docs/Benchmarking.html (visited on 03/11/2025).

[28] J. Jenkov. "JMH - Java Microbenchmark Harness." (), [Online]. Available: http://jenkov.com/tutorials/java-performance/jmh.html (visited on 03/12/2025).

[29] M. Raasveldt and H. Mühleisen, "DuckDB: An Embeddable Analytical Database," in *Proceedings of the 2019 International Conference on Management of Data*, ser. SIGMOD '19, New York, NY, USA: Association for Computing Machinery, Jun. 25, 2019, pp. 1981–1984, ISBN: 978-1-4503-5643-5. DOI: 10.1145/3299869.3320212.